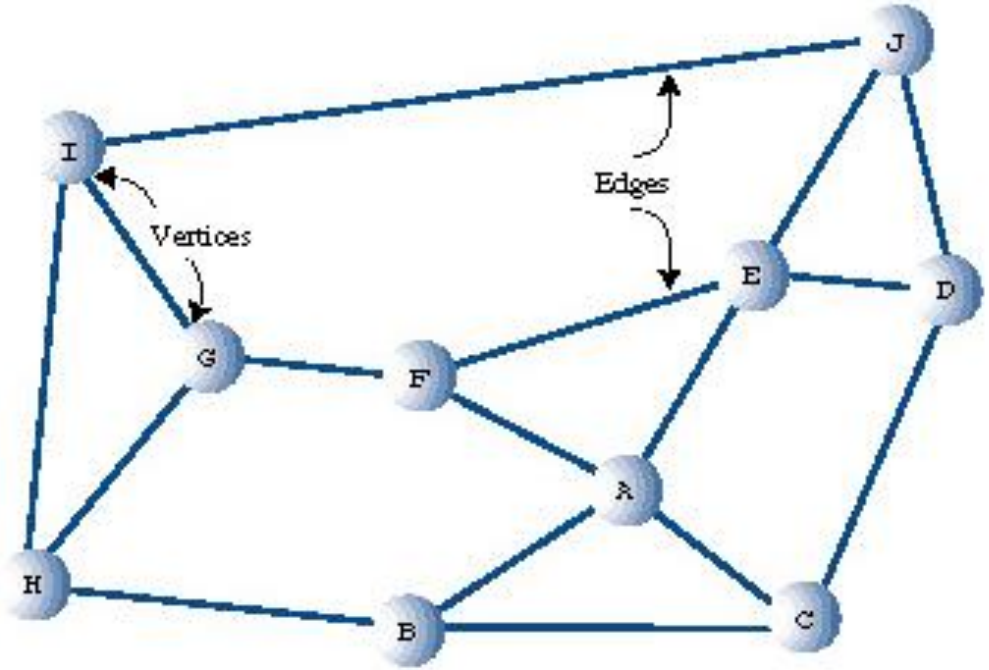
Graphs

Contents

- Definition
- Implementation
- Depth First Search
- Breadth First Search
- Topological sorting
- Dijkstra Algorithm
- Connectivity Problem

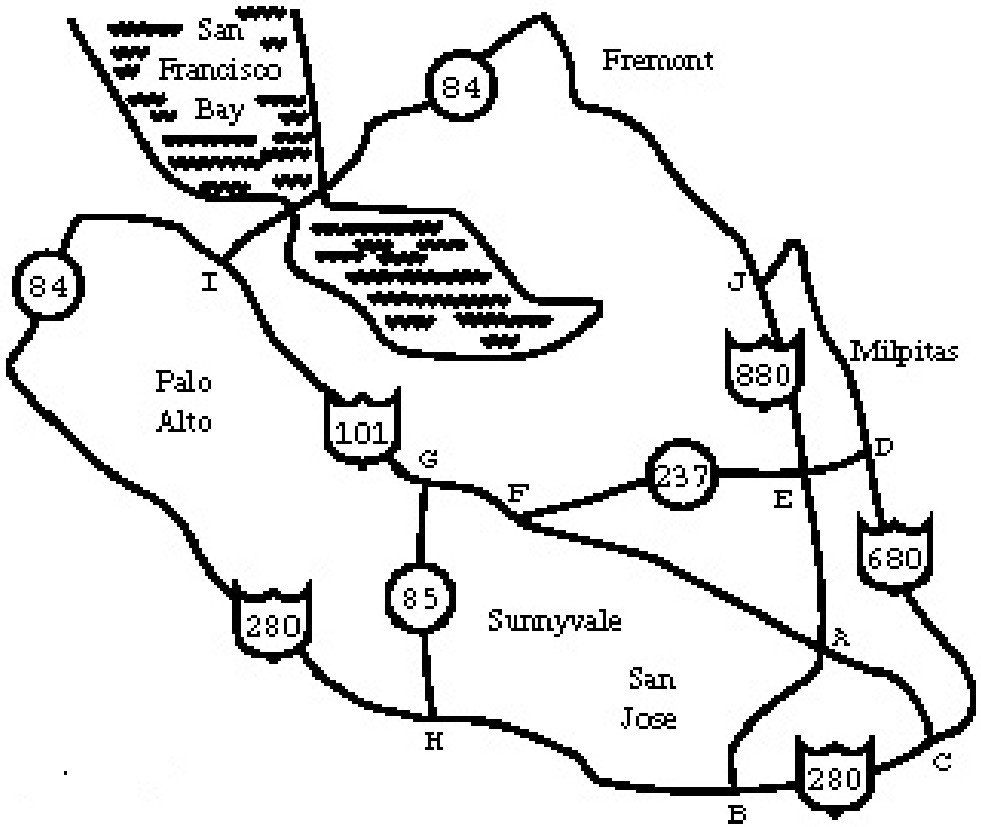
Graphs

- Graph is a data structure which represent relationships between entities
  - ***Vertices*** represent entities
  - ***Edge***



Example

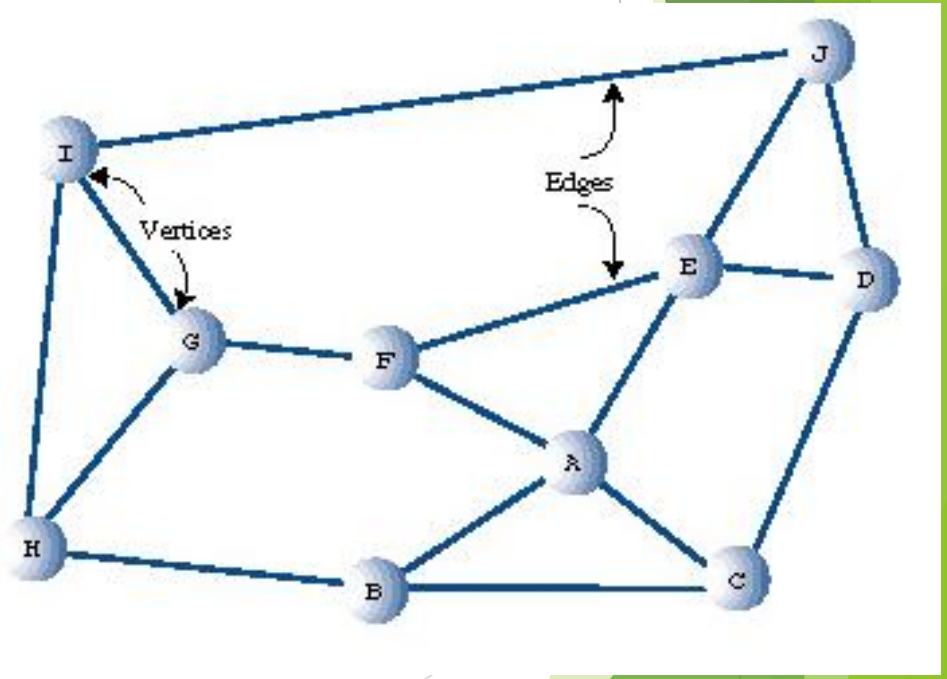
- The graph on the previous page could be used to model San Jose fr



Adjacency

- Two vertices are ***adjacent*** to one another if they are connected by a single edge

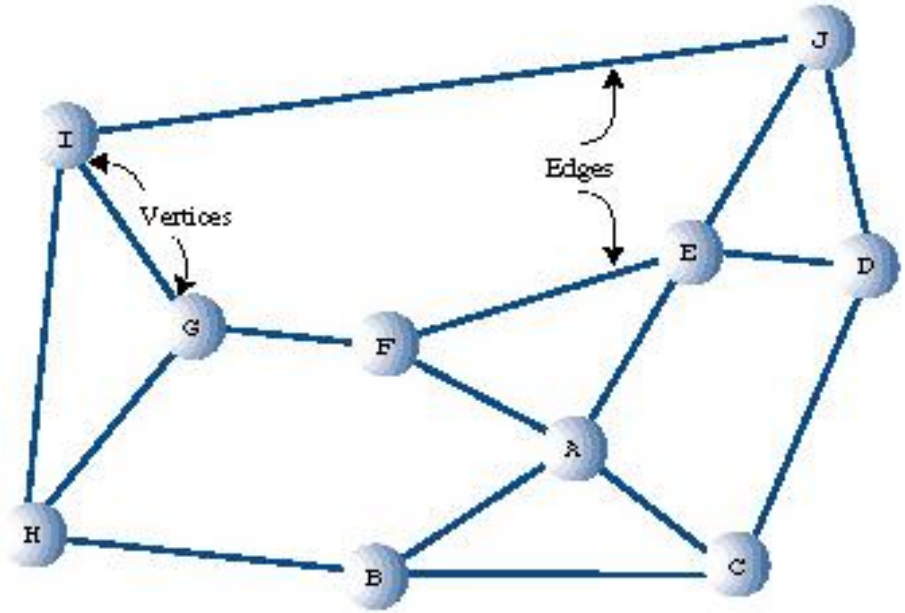
- For example:
  - I and G are adjacent
  - A and C are adjacent
  - I and F are not adjacent



- Two adjacent nodes are considered ***neighbors***

Path

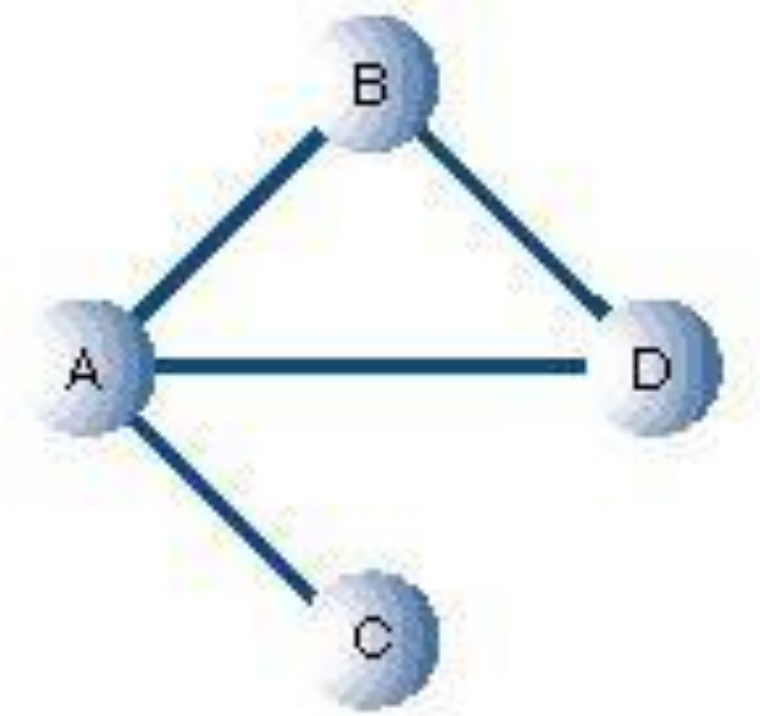
- A ***path*** is a



- Paths in this graph include:
  - BAEJ, CAFG, HFEJDCAB, HIJDCBAFE, etc.

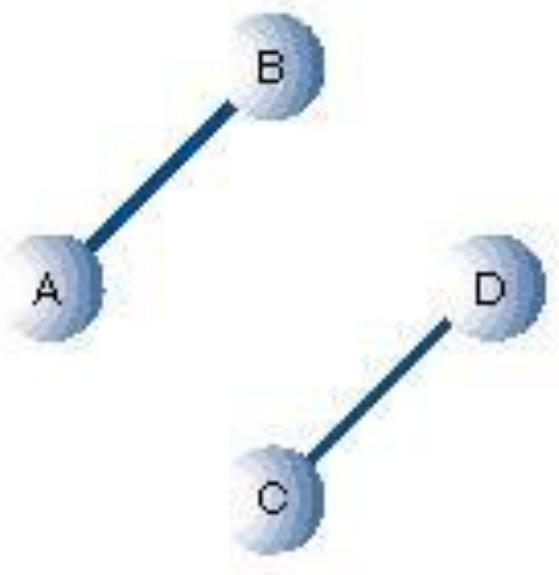
Connected Graphs

- A graph is connected if there is at least one path from every vertex to every other vertex



Unconnected Graph

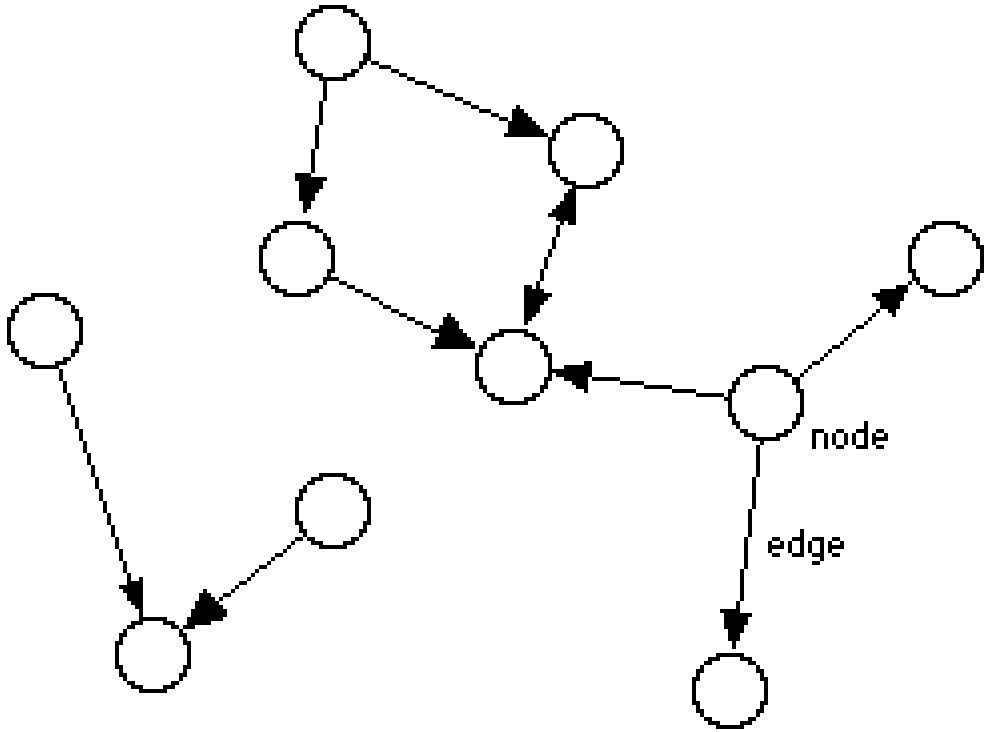
- An ***unconnected graph components***:



- Connected components of this graph are:
  - AB, and CD
- We'll be working with connected graphs

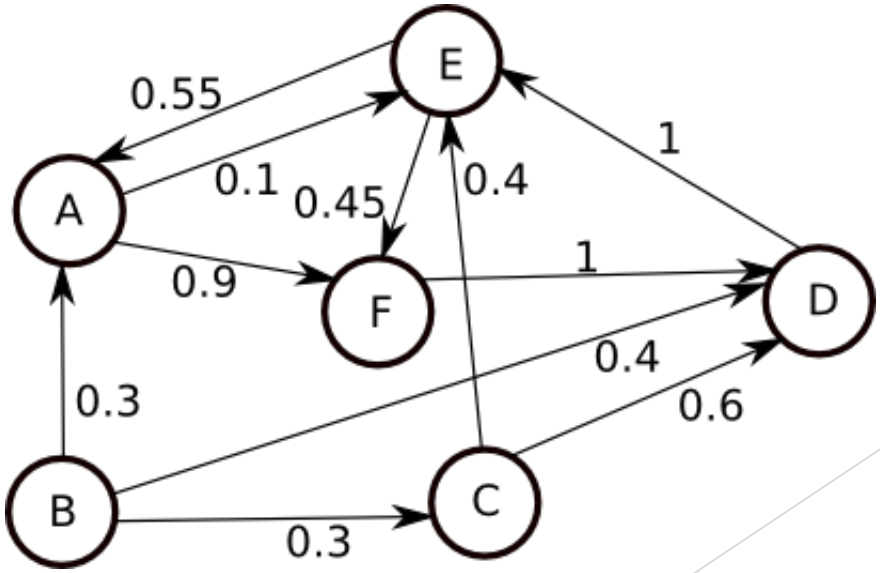
Directed Graphs

- A graph where edges have directions
  - Usually designated by an arrow



Weighted Graphs

- A graph where edges have weights, which quantifies the relationship
  - For example, you may assign path distances between cities
  - Or airline costs
- These graphs can be directed or undirected

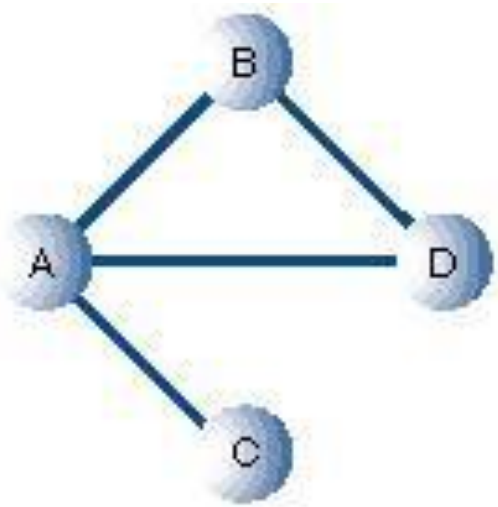


Vertices: Java Implementation

- We can represent a vertex as a Java class with:
  - Character data
  - A boolean data member to check if it has been visited
- Now we need to specify edges.
  - But in a graph, we don't know how many there will be!
- We can do this using either an adjacency ***matrix*** or an adjacency ***list***. We'll look at both.

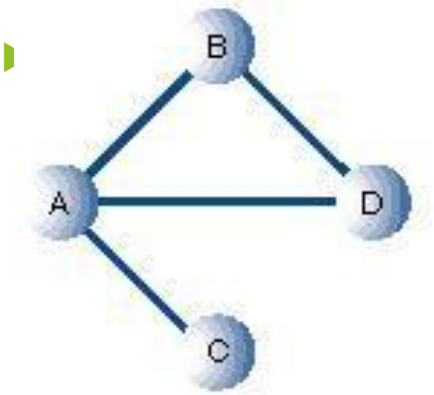
Adjacency Matrix

- An adjacency matrix for a graph with n nodes, is size n x n
  - Position (i, j) contains a 1 if there is an edge connecting node i with node j
  - Zero otherwise



	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	0
D	1	1	0	0

Redundant?



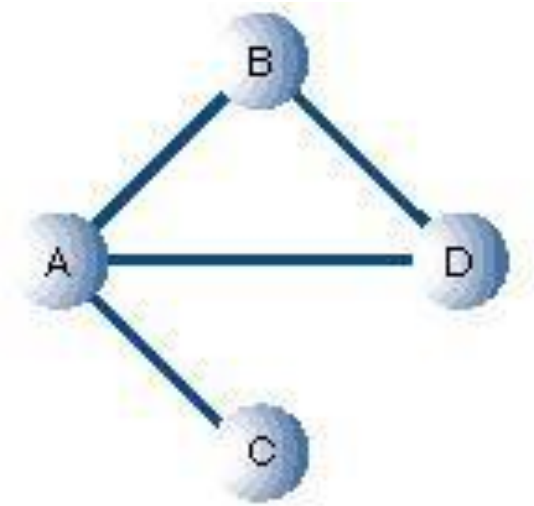
it r

	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	0
D	1	1	0	0

- Why store two pieces of information for the same edge?
  - i.e., (A, B) and (B, A)
- Unfortunately, there's no easy way around it
  - Because edges have no direction
  - No concept of 'parents' and 'children'

Adjacency List

- An array of linked lists
  - Index by vertex, and obtain a linked list of neighbors
- Here is the same graph. with its adiacencv list:



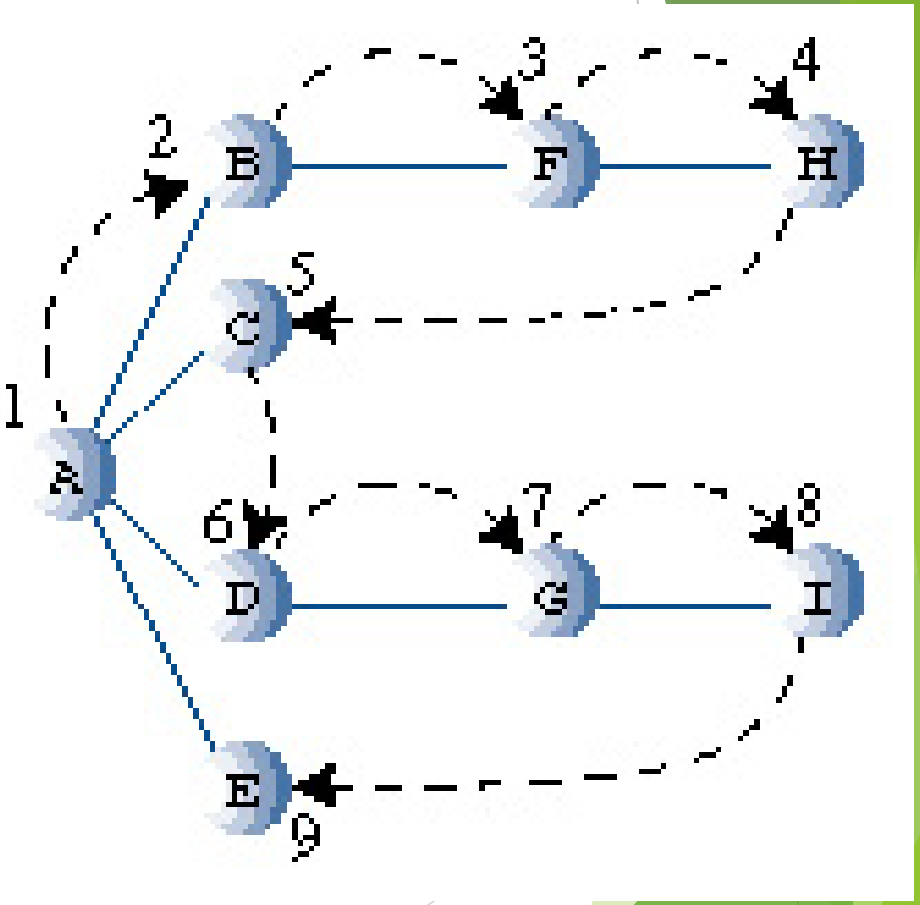
Vertex	List Containing Adjacent Vertices
A	B—>C—>D
B	A—>D
C	A
D	A—>B

Application: Searches

- A fundamental operation for a graph is:
  - Starting from a particular vertex
  - Find all other vertices which can be reached by following paths
- Example application
  - How many towns in the VN can be reached by train from Da Lat?
- Two approaches
  - Depth first search (DFS)
  - Breadth first search (BFS)

Depth First Search (DFS)

- Idea
  - Pick a starting point
  - Follow a path to unvisited vertices, as long as you can until you hit a dead end
  - When you hit a dead end, go back to a previous spot and hit unvisited vertices
  - Stop when every path is a dead end



Depth First Search (DFS)

- Algorithm
  - Pick a vertex (call it A) as your starting point
  - Visit this vertex, and:
    - Push it onto a stack of visited vertices
    - Mark it as visited (so we don't visit it again)
  - Visit any neighbor of A that hasn't yet been visited
    - Repeat the process
  - When there are no more unvisited neighbors
    - Pop the vertex off the stack
  - Finished when the stack is empty

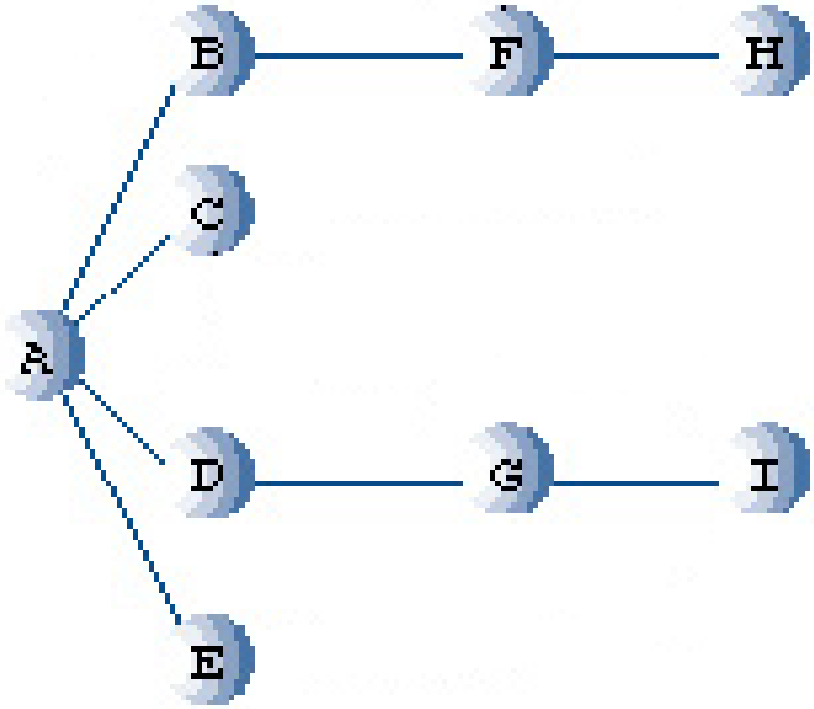
- Note: We get as far away from the starting point until we reach a dead end, then pop (can be applied to mazes)

TABLE 13.3 Stack Contents During Depth-First Search

Event	Stack
Visit A	A
Visit B	AB
Visit F	ABF
Visit H	ABFH
Pop H	ABF
Pop F	AB
Pop B	A
Visit C	AC
Pop C	A
Visit D	AD
Visit G	ADG
Visit I	ADGI
Pop I	ADG
Pop G	AD
Pop D	A
Visit E	AE
Pop E	A
Pop A	
Done	

Example

- Start from A, and execute depth first search on this graph, showing the contents of the stack at each step
  - Every



Depth First Search: Complexity

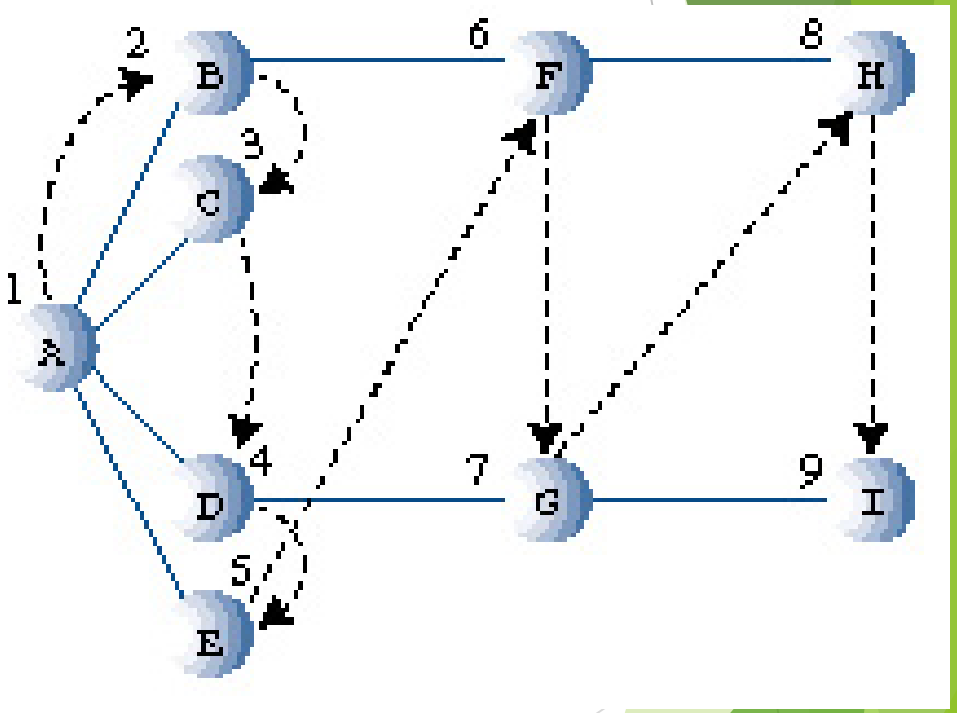
- Let |V| be the number of vertices in a graph
- And let |E| be the number of edges

- In the worst case, we visit every vertex and every edge:
  - O(|V| + |E|) time

- At first glance, this doesn't look too bad
  - But remember a graph can have lots of edges!
  - Worst case, every vertex is connected to every other:
    - $(n-1) + (n-2) + \ldots + 1 = O(n^2)$
  - So it can be expensive if the graph has many edges

Breadth First Search (BFS)

- Same application as DFS; we want to find all vertices which we can get to from a starting point, call it A
- However this time, instead of going as far as possible until we find a dead end, like DFS
  - We visit all the closest vertices first
  - Then once all the closest vertices are visited, branch further out

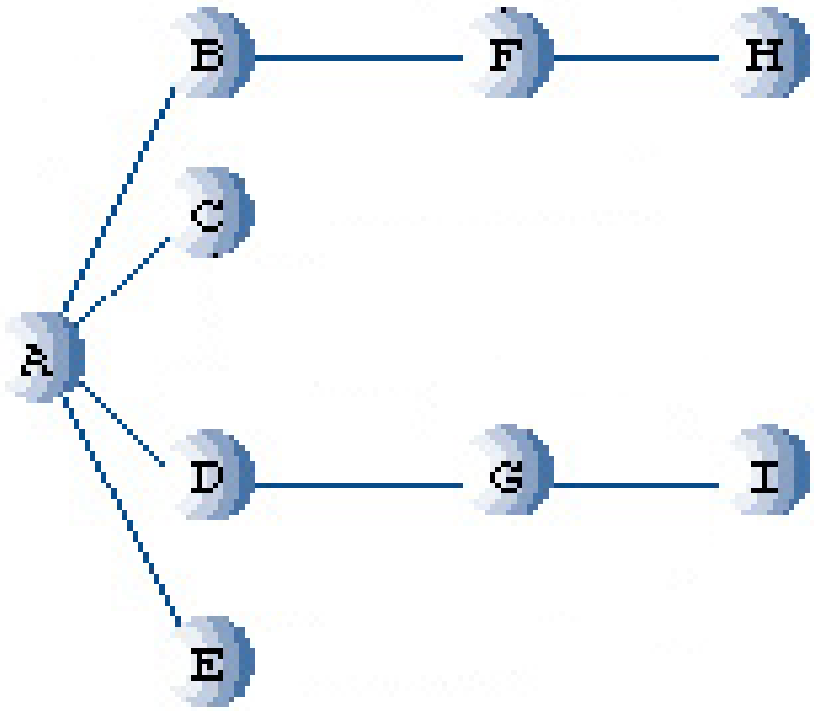


Breadth First Search (BFS)

- We're going to use a queue instead of a stack!
- Algorithm
  - Start at a vertex, call it current
  - If there is an unvisited neighbor, mark it, and insert it into the queue
  - If there is not:
    - If the queue is empty, we are done, otherwise:
    - Remove a vertex from the queue and set current to that vertex, and repeat the process

Example

- Start from A, and execute breadth first search on this graph, showing the contents of the queue at each step
  - Every



Breadth First Search: Complexity

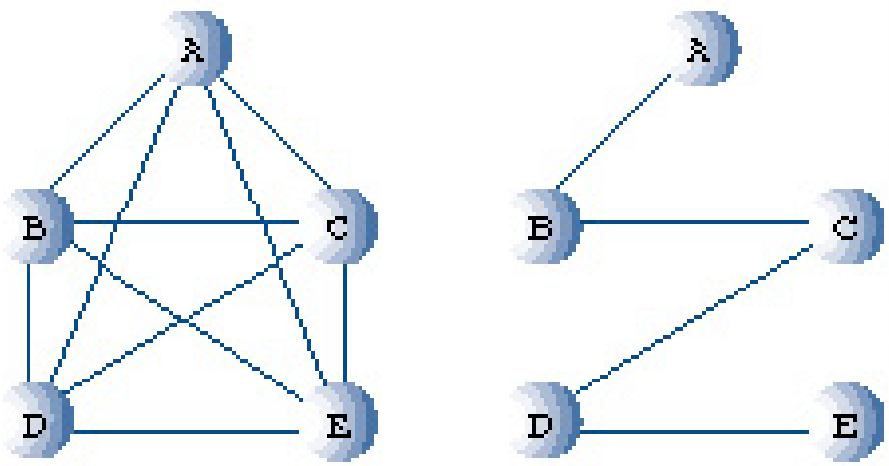
- Let |V| be the number of vertices in a graph
- And let |E| be the number of edges

- In the worst case, we visit every vertex and every edge:
  - O(|V| + |E|) time
  - Same as DFS

- Again, if the graph has lots of edges, you approach quadratic run time which is the worst case

Minimum Spanning Trees (MSTs)

- On that note of large numbers of edges slowing down our precious search algorithms:
  - Let's look at MSTs, which can help ameliorate this problem
- It would be nice to take a graph and reduce the number of edges to the minimum number required to span all vertices:



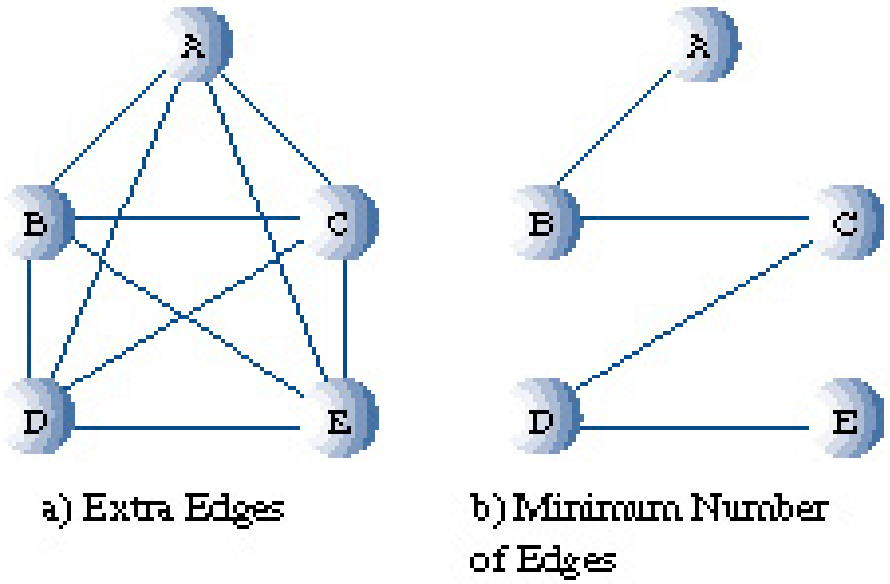
a) Extra Edges

b) Minimum Number of Edges

What's the number of edges now?

We've done it already…

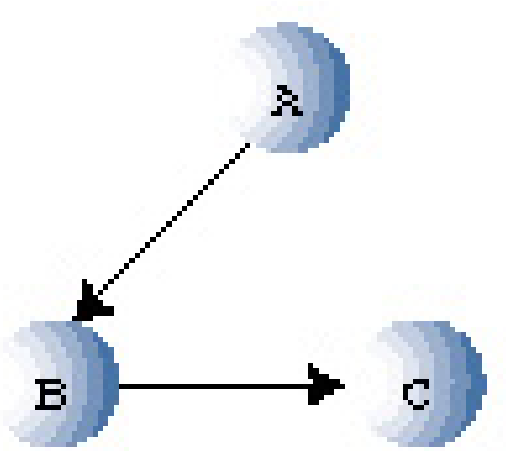
- Actually, if you execute DFS you've already computed the MST!



- Think about it: you follow a path for as long as you can, then backtrack (visit every vertex at most once)
  - You just have to save edges as you go

Directed Graphs

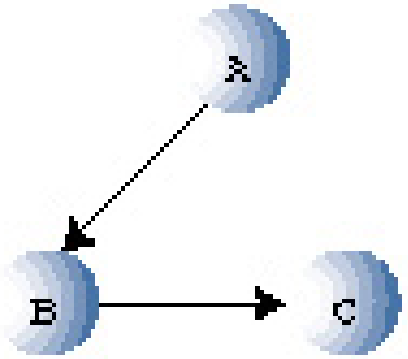
- A directed graph is a graph where the edges have direction, signified by arrows:



- This will simplify the adjacency matrix a bit…

Adjacency Matrix

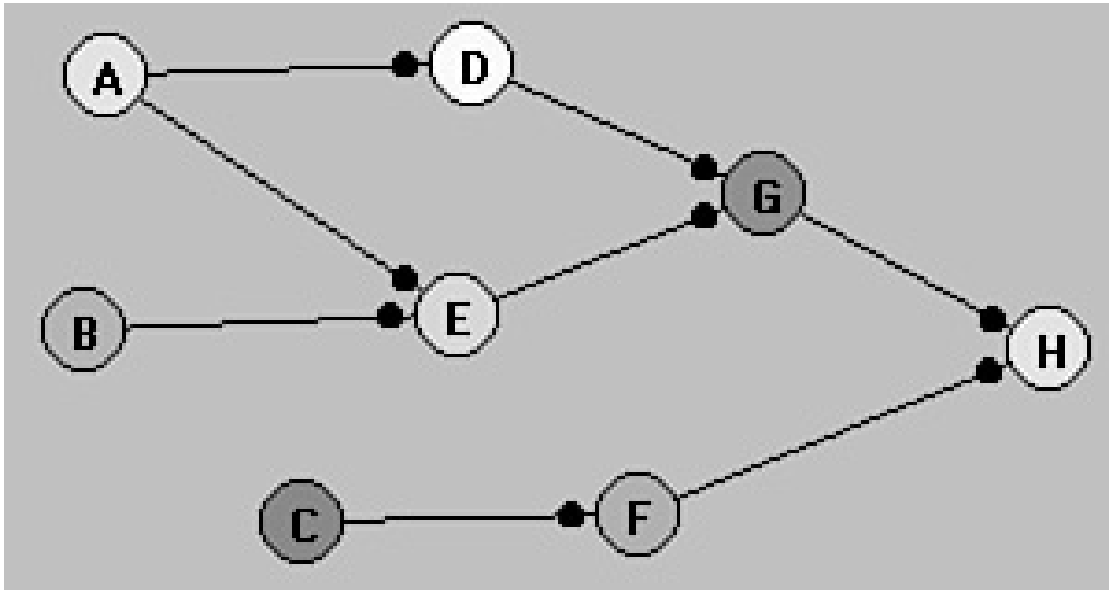
- The adjacency matrix for this graph does not contain redundant entries
  - Because now each edge has a source and a sink
  - So entry (i, j) is only set to 1 if there is an edge going from i to j
    - 0 otherwise



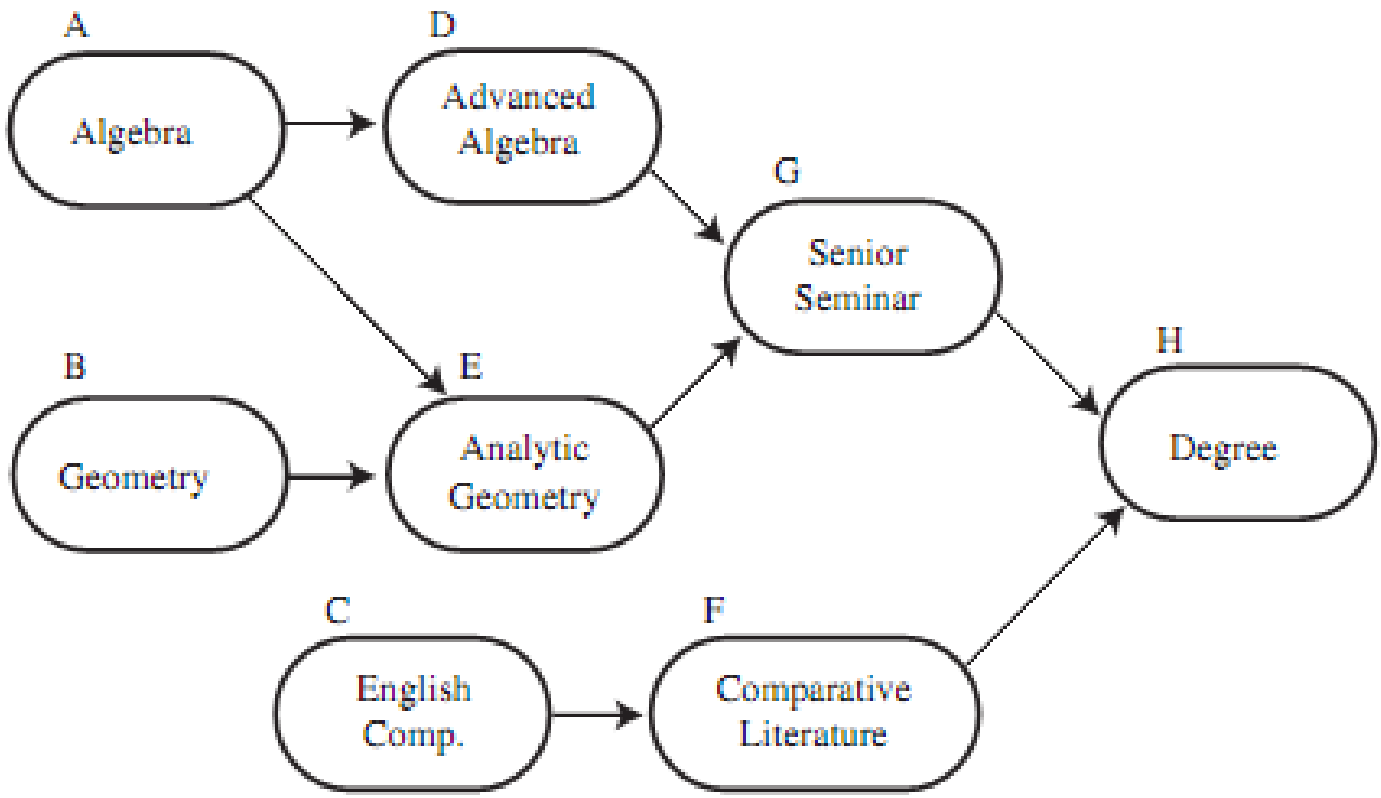
	A	B	C
A	0	1	0
B	0	0	1
C	0	0	0

Topological Sort

- Only works with DAGs (Directed Acyclic Graphs)
  - That is if the graph has a cycle, this will not work



- Idea: Sort all vertices such that if a vertex's successor always appears after it in a list (application: course prerequisites)

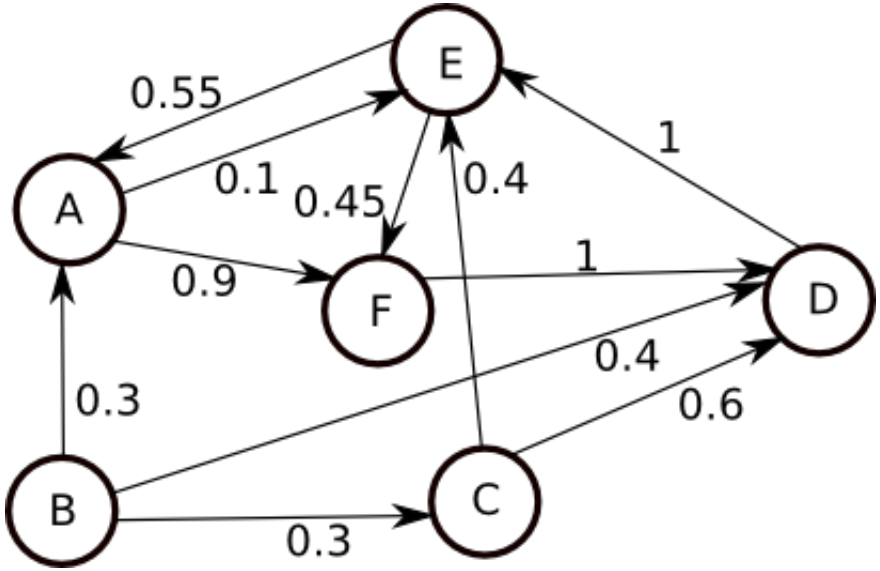

A
Algebra
D
Advanced Algebra
G
Senior Seminar
B
Geometry
E
Analytic Geometry
H
Degree
C
English Comp.
F
Comparative Literature

Topological Sort

- Algorithm
  - Find a vertex that has no successors
  - Delete this vertex from the graph, and insert its label at the beginning of a list
  - Repeat until every vertex is gone
- Works because if a vertex has no successors, it must be the last one (or tied for last) in the topological ordering
  - As soon as it's removed, at least one of the remaining vertices must not have successors
    - Because there are no cycles!
  - Graph does NOT have to be fully connected for this to work!

Weighted Graphs

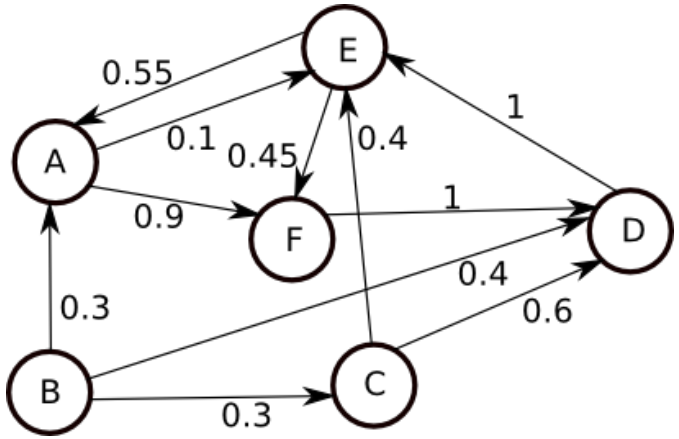
- Once again, a graph where edges have weights, which quantifies the relationship
- These graphs can be directed or undirected



Weighted Graph: Adjacency List

- The adjacency list for a weighted graph contains edge weights
  - Instead of 0 and 1
- If there is no edge connecting vertices i and j, a weight of INFINITY is used (not 0!)
  - Because ‘0’ can also be a weight
  - Also most applications of weighted graphs are to find minimum spanning trees or shortest path (we’ll look at this)
- Also remember if the graph is undirected, redundant information should be stored

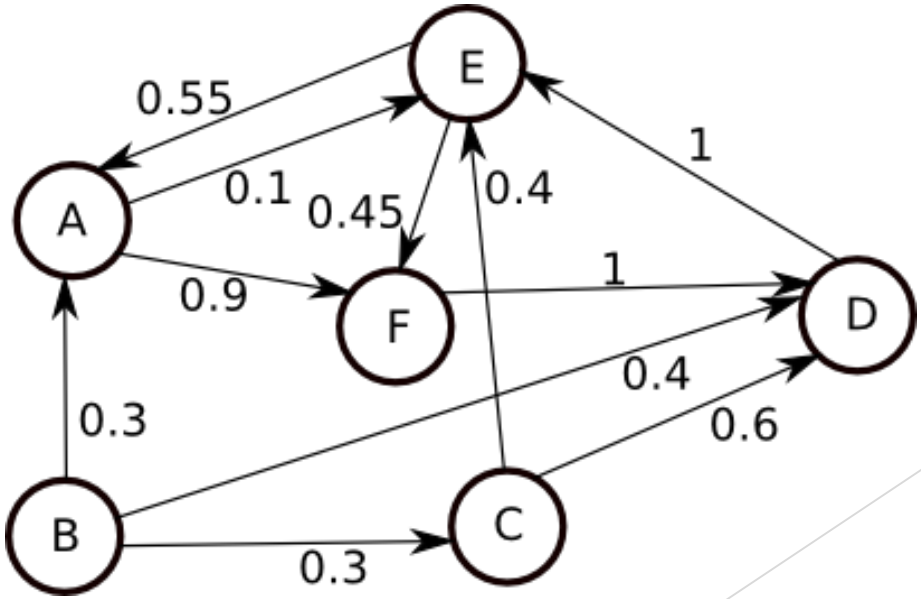
Weighted Graph: Adjacency List



	A	B	C	D	E	F
A	INF	INF	INF	INF	0.1	0.9
B	0.3	INF	0.3	0.4	INF	INF
C	INF	INF	INF	0.6	0.4	INF
D	INF	INF	INF	INF	1	INF
E	0.55	INF	INF	INF	INF	0.45
F	INF	INF	INF	1	INF	INF

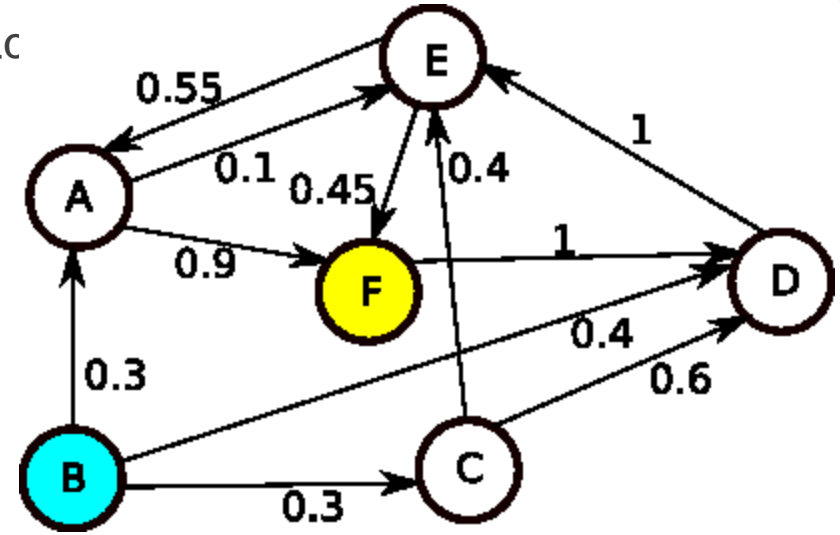
Dijkstra's Algorithm

- Given a weighted graph, find the shortest path (in terms of edge weights) between two vertices in the graph
- Numerous applications
  - Cheapest airline fare between departure and arrival cities
  - Shortest driving distance in terms of mileage



Dijkstra's Algorithm

- Suppose in the graph belc
  from B to F



- Idea: Maintain a table of the current shortest paths from B to all other vertices (and the route it takes)
  - When finished, the table will hold the shorest path from B to all other vertices

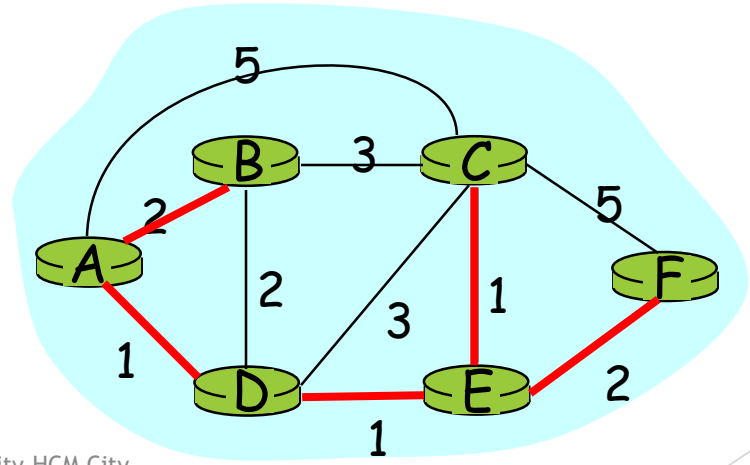
Dijsktra's Algorithm

```
Initialization:
  N = {A}
  for all nodes v
    if v adjacent to A
      then D(v) = c(A,v)
      else D(v) = infty

Loop
   find w not in N such that D(w) is a minimum
   add w to N
   update D(v) for all v adjacent to w and not in N:
       D(v) = min( D(v), D(w) + c(w,v) )
   /* new cost to v is either old cost to v or known
    shortest path cost to w plus cost from w to v */
until all nodes in N
```

Dijkstra's algorithm example

Step	start N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	infinity	infinity
1	AD	2,A	4,D		2,D	infinity
2	ADE	2,A	3,E			4,E
3	ADEB		3,E			4,E
4	ADEBC					4,E
5	ADEBCF					



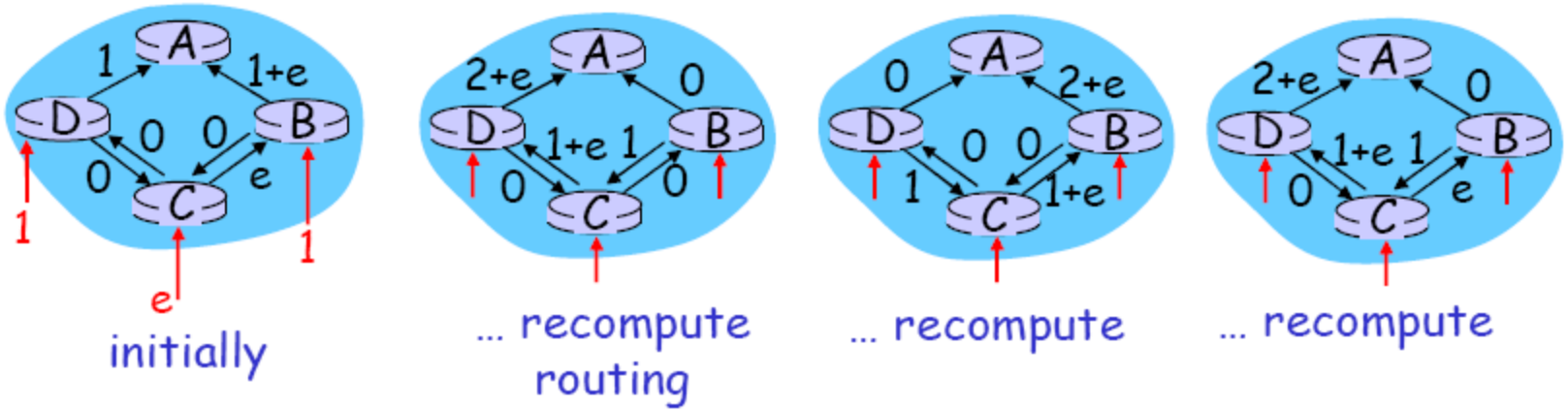
Dijkstra's algorithm, discussion

Algorithm complexity: n nodes

- each iteration: need to check all nodes, w, not in N
- $n(n+1)/2$ comparisons: $O(n^2)$
- more efficient implementations possible: $O(n\log n)$

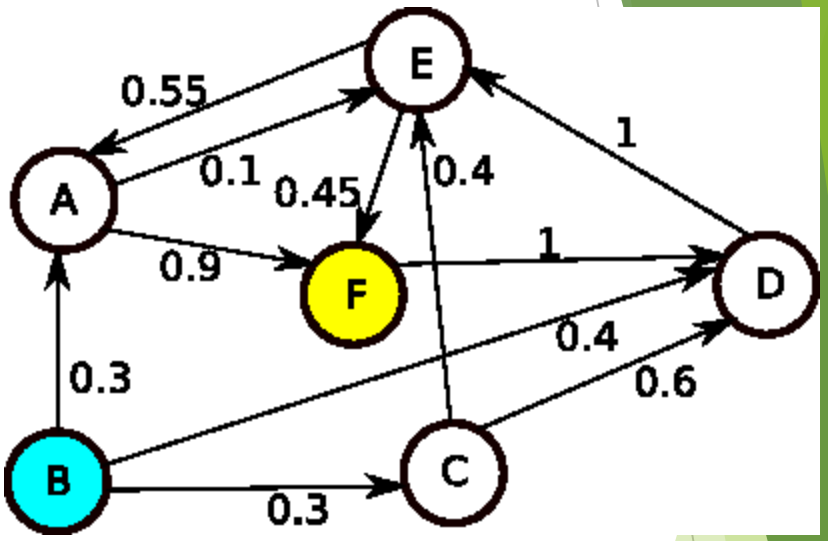
Oscillations possible:

- e.g., link cost = amount of carried traffic



Dijkstra's Algorithm

- Here is our initial graph:

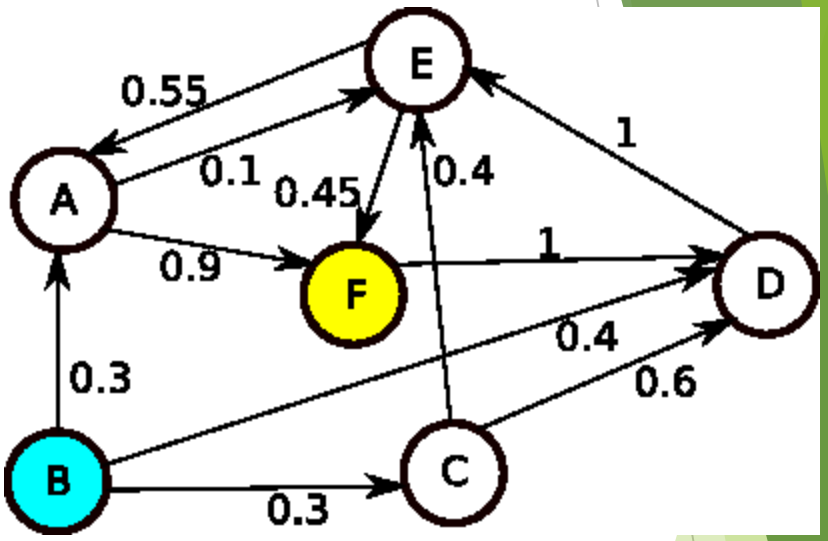


- And here is our table:

A	C	D	E	F
INF	INF	INF	INF	INF

Step 1

- Take all edges leaving B, and put their weights in the table
  Along with the source vertex

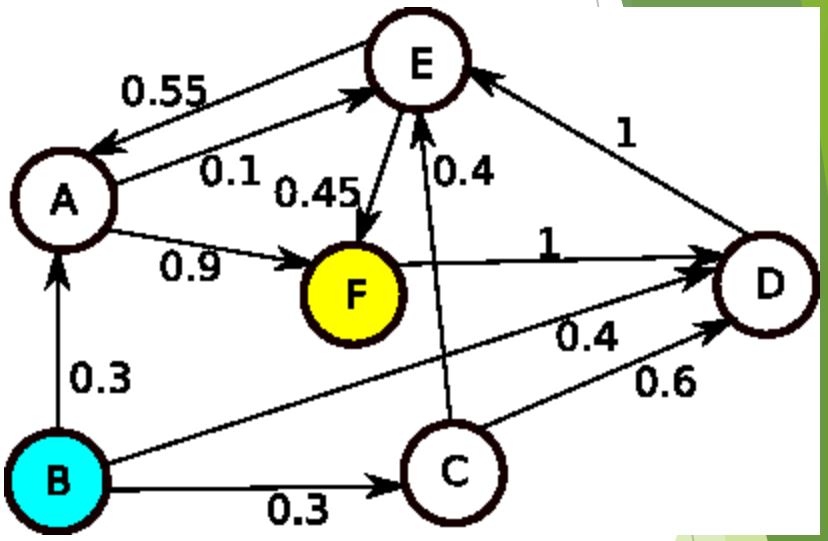


- And here is our table:

A	C	D	E	F
0.3 (B)	0.3 (B)	0.4 (B)	INF	INF

Step 2

- Pick the edge with the smallest weight and mark it as the shortest path from B (How do we know that?)

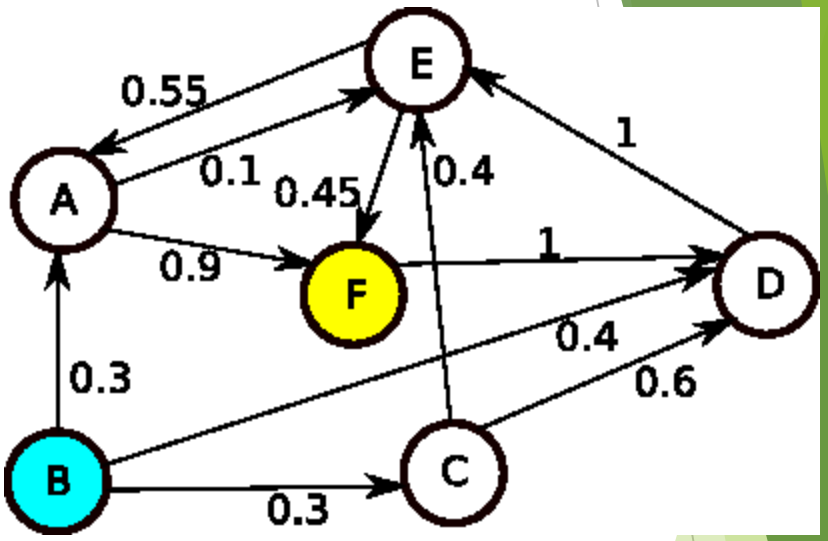


- And here is our table:

A	C	D	E	F
0.3* (B)	0.3 (B)	0.4 (B)	INF	INF

Step 3

- Now choose one of the edges with minimal weight and repeat the process (explore adj. vertices and mark their total weight)



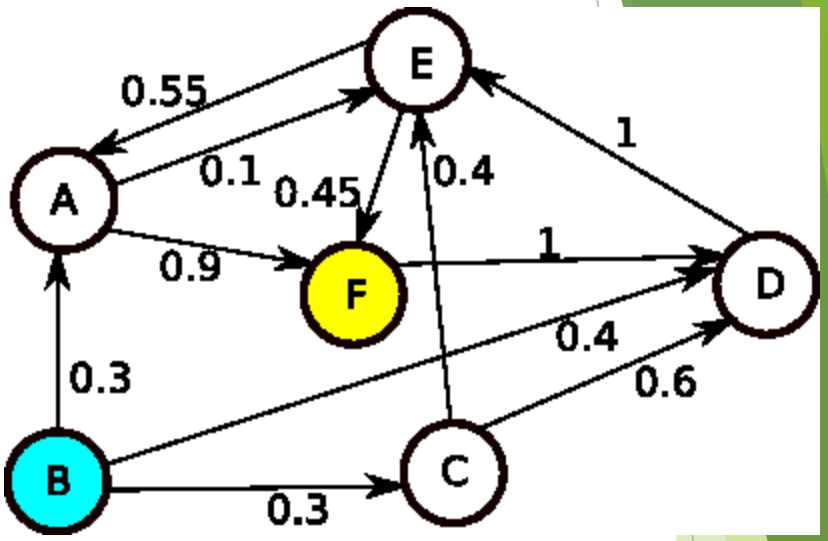
- And here is our table:

A	C	D	E	F
0.3* (B)	0.3 (B)	0.4 (B)	INF	INF

Step 4

- In this case, we'll look at A
  - Explore adjacent vertices
  - Enter the total weight

    from B to those vertices
    - IF that weight is smaller than
    - the current entry in the table
    - Ignore the ones marked (*)

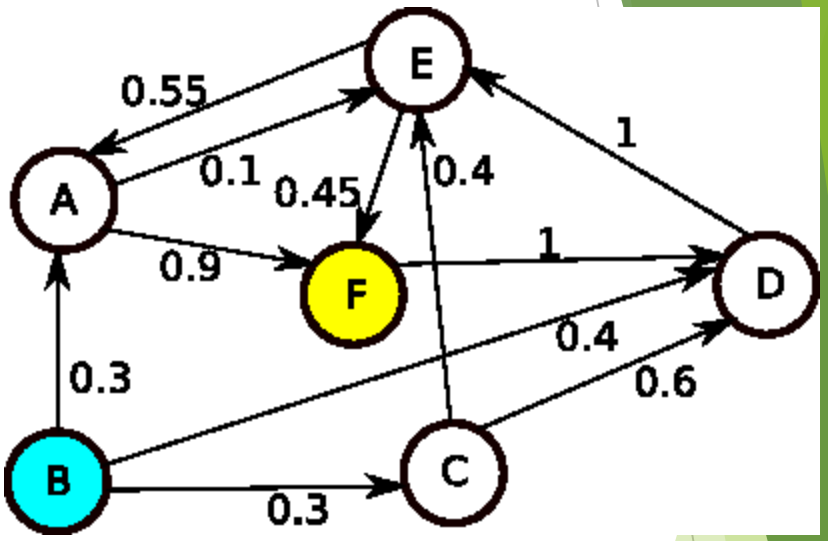


- So here is our table:

A	C	D	E	F
0.3* (B)	0.3 (B)	0.4 (B)	0.4 (A)	1.2 (A)

Step 5

- Now, A is marked and
- we've visited its neighbors
  - So pick the lowest entry in the table (in this case C) and repeat the process

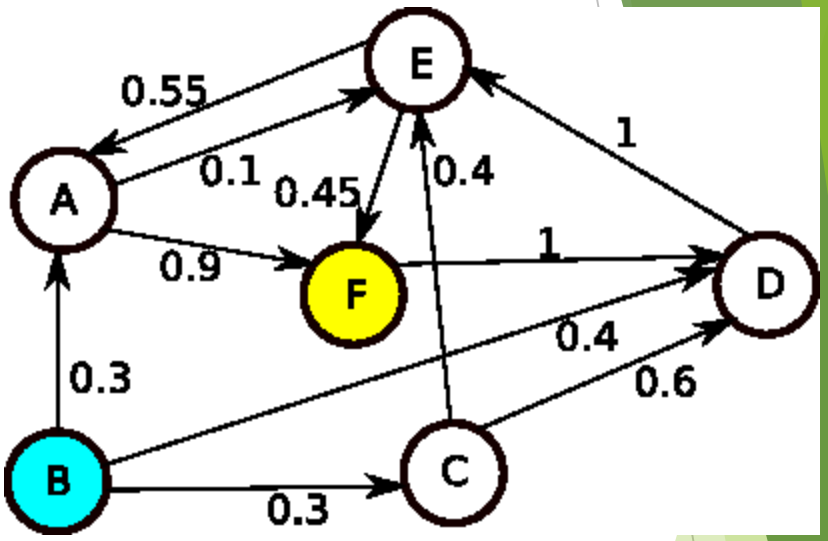


- So here is our table:

A	C	D	E	F
0.3* (B)	0.3* (B)	0.4 (B)	0.4 (A)	1.2 (A)

Step 6

- Visit C's neighbors that
- are unmarked
  - Insert their total weight
  - into the table, IF it's
  - smaller than the current
  - entry

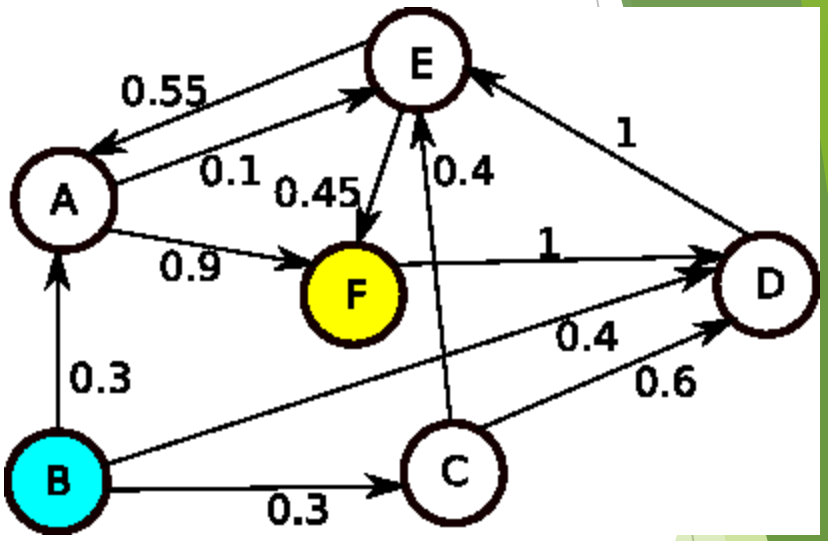


- So in fact, nothing changes in the table:

A	C	D	E	F
0.3* (B)	0.3* (B)	0.4 (B)	0.4 (A)	1.2 (A)

Step 7

- Now we visit D
  - Which only contains
  - one edge to E
  - 0.4+1 = 1.4, which
  - is larger than 0.4

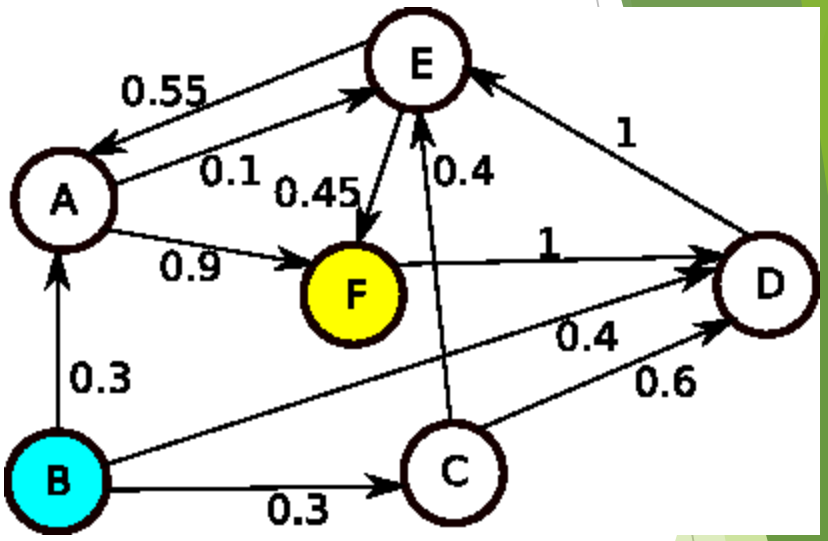


- So again, nothing changes in the table:

A	C	D	E	F
0.3* (B)	0.3* (B)	0.4* (B)	0.4 (A)	1.2 (A)

Step 8

- Now we visit E
  - Has two outgoing edges
    - One to A (marked, ignore)
    - One to F, which changes
    - the table to
      - 0.4 + 0.45 = 0.85
      - Which is smaller than the
      - current entry, 1.2

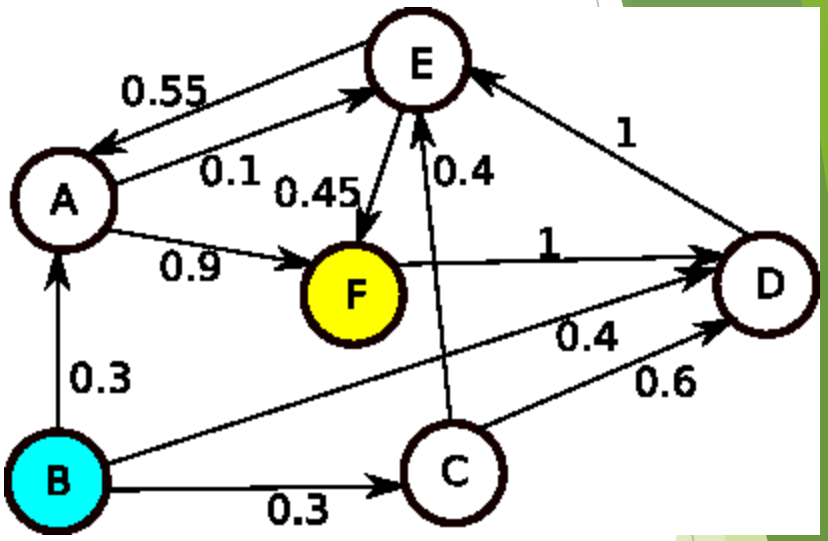


- So the table changes, we found a shorter path to F:

A	C	D	E	F
0.3* (B)	0.3* (B)	0.4* (B)	0.4* (A)	0.85 (E)

Step 9

- Only one vertex left, so
- we're actually finished
  - Shortest path can be
  - obtained by starting from
  - the destination entry and
  - working backwards
    - F <- E <- A <- B

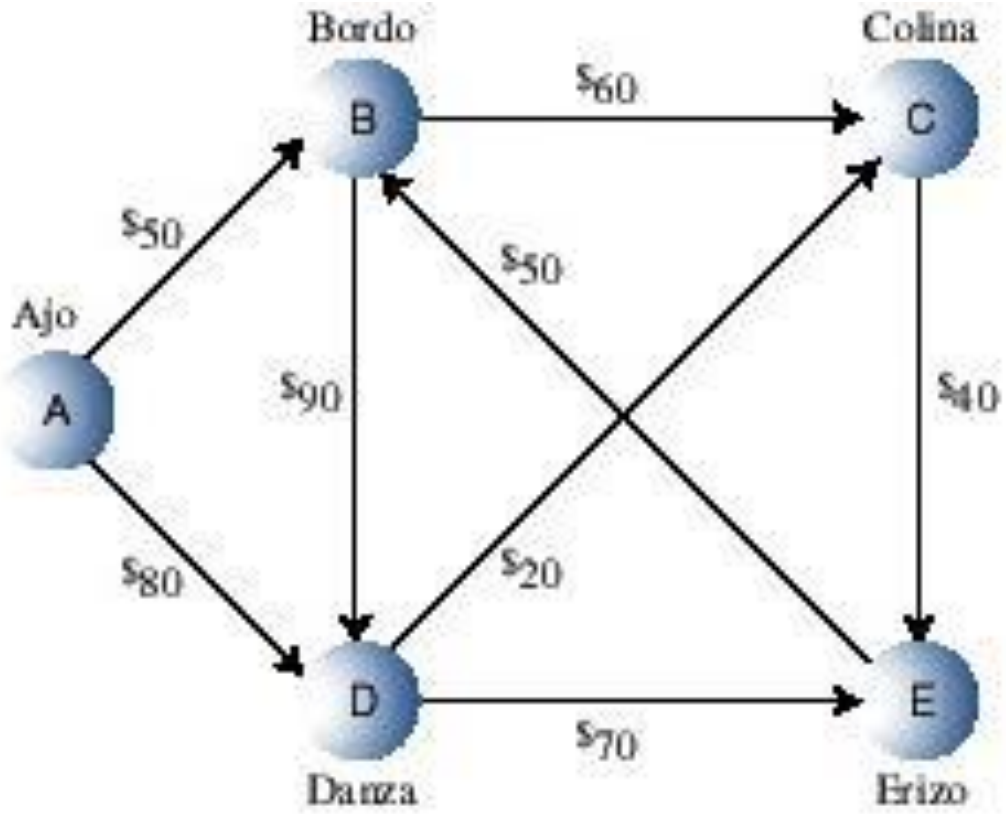
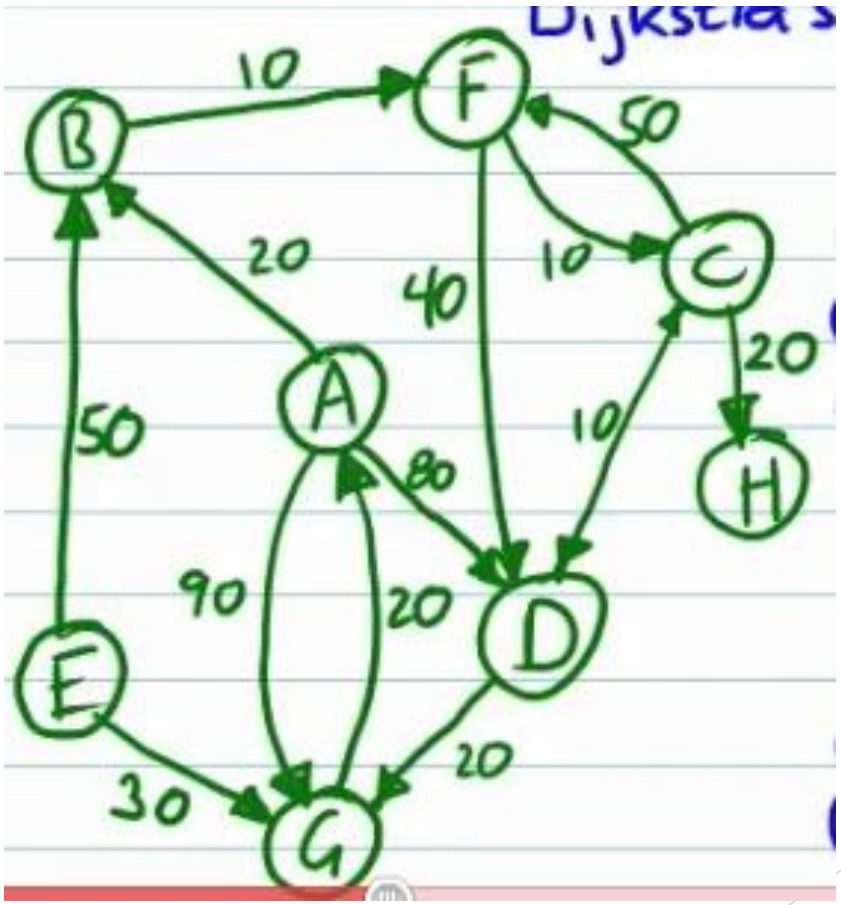


- Shortest path from B to F is: B -> A -> E -> F
  - Total weight: 0.85

A	C	D	E	F
0.3* (B)	0.3* (B)	0.4* (B)	0.4* (A)	0.85* (E)

Example

- Let’s



Exercise